Real-Time Multi-Behavior Detection of Group-Housed Pigs on Edge Devices: Class-Imbalance-Aware Sampling and a Lightweight FasterNet Backbone

[Authors / affiliations to be filled]

Abstract

Automated monitoring of group-housed pigs needs detectors that are accurate on rare, welfare-relevant behaviors and fast enough for low-cost edge hardware. We built a real-time multi-behavior detection framework on YOLOv11n and validated it on a Jetson Nano. Three things make the framework practical. First, a class-imbalance-aware sampling strategy based on frequency-capped offline oversampling improves rare-class detection without touching the network: the average precision of the low-frequency behavior active rises from 0.526 to 0.639 on the test set, and overall recall improves by 2.1 points on the validation set. Second, a FasterNet backbone substitution, combined with an identity-preserving integration strategy that transfers pre-trained weights by index-aligned remapping, cuts parameters by 4.4% while accuracy stays statistically comparable to the baseline across three repeated runs (test mean average precision at IoU 0.5 (mAP50) of 0.590 ± 0.009 vs. a three-seed baseline of 0.606 ± 0.008); interestingly, it is also the fastest of all models we benchmarked (117.6 frames per second (FPS) on an RTX 3090). Third, on-device validation on the Nano shows real-time operation (19.7 FPS at 640×640; 33.3 FPS at 480×480) at roughly 5 W total power. We also report a two-level generalization analysis—an unseen-sequence stress test and an independent farm dataset—that honestly delimits where the framework works and where it does not.
Keywords: pig behavior detection; class imbalance; FasterNet; edge computing; Jetson Nano; precision livestock farming

1. Introduction

Pig behavior tells us almost everything we can know about an animal without touching it. Feeding and drinking reflect metabolism and homeostasis; resting postures indicate comfort and health status; locomotion and activity levels reveal vitality; and agonistic interactions—fighting and bullying—signal resource competition, stress, and welfare risks [1, 2]. Departures from normal behavioral patterns are often the earliest visible signs of stress or disease, which makes continuous behavior monitoring one of the most practical instruments of precision livestock farming (PLF) [3]. Manual inspection cannot scale to commercial herds, so camera-based, non-contact monitoring has become the default approach.

For such monitoring to be economically viable, however, detection has to run on inexpensive edge devices inside the barn. A commercial farm may operate hundreds of pens; streaming all of that video to a cloud server is impractical over rural networks, and the recurring cost of cloud inference dwarfs the one-time cost of an edge board. Latency matters as well: an alert about a fight that arrives minutes late is of little use. This is why the dominant engineering trend is to bring the detector to the data, not the other way around.

YOLO-family detectors [4] are the workhorse of livestock behavior detection because they combine real-time speed with competitive accuracy. Yet three practical issues keep recurring in the literature. First, behavioral data are strongly imbalanced: resting and exploratory behaviors dominate, while drinking and brief social interactions are rare. Uniformly sampled training is therefore biased toward frequent classes—which are usually the least informative for welfare alerting. Second, most published livestock detectors are evaluated only on server GPUs, so their behavior on memory- and power-constrained edge hardware is largely undocumented; an accuracy report does not tell us whether the model will actually run in a pig barn. Third, evaluations are almost always performed on frames drawn from the same video sequences as the training data. Adjacent frames are near-duplicates, so reported accuracy is optimistic, and cross-pen, cross-day, or cross-farm generalization remains unmeasured.

We target these three gaps with a framework that combines data-level re-balancing, a lightweight backbone, and on-device validation. Our contributions are:

(1) A class-imbalance-aware sampling strategy (frequency-capped offline oversampling) that improves rare-behavior detection without modifying the network or the loss, together with a sensitivity analysis over the duplication cap.

(2) A lightweight backbone substitution (FasterNet blocks) paired with an identity-preserving integration strategy. In our early attempts, inserting new gating modules into the pre-trained network corrupted the pre-trained feature distribution and cost 3–7 mAP points; keeping the channel flow intact and transferring weights by index-aligned remapping avoids that damage entirely—accuracy is preserved while parameters drop by 4.4%.

(3) A deployment validation on Jetson Nano (19.7–33.3 FPS at ≈5 W) and a two-level generalization analysis (unseen sequences; an independent farm dataset) that delimit the operating envelope of the framework.

2. Related Work

2.1 Vision-Based Livestock Behavior Detection

YOLO-based detectors have been applied to pig detection, individual tracking, posture recognition, and behavior analysis. For group-housed pigs, Tu et al. [5] combined YOLOv5s/YOLOX-S detectors with an improved DeepSORT tracker to track individual behaviors such as lying, feeding, and standing; Li et al. [6] paired YOLOX with the SCTS-SlowFast spatio-temporal module for joint detection and recognition of multiple behaviors; and more recent work proposed improved YOLOv8n and YOLOv11 variants—typically by inserting attention modules or re-designing feature-fusion paths—for standing, lying, feeding, drinking, and biting behaviors. Earlier, Alameer et al. [7] had already recognized postures and drinking behaviour for compromised-health monitoring from overhead barn footage. These works establish feasibility, but they largely assume server-side inference and balanced class distributions, and few report per-class results on rare behaviors. Two very recent studies push the same deployment agenda with different emphases. PBR-YOLO [27] combines a GhostNet backbone, a FasterNet block with efficient multi-scale attention, and a lightweight multi-path head for eight piglet behaviors, reporting 15.8 ms per frame with TensorRT on a Jetson Orin NX—an attention-augmented design demonstrated on a board far more powerful than the Nano-class hardware we target. Rahman et al. [28] trim YOLO11n for the single posture–nursing task in farrowing crates (removing the small-object detection head) and accelerate it with TensorRT, evaluating on a random image-level split that they explicitly frame as within-facility performance. Class-imbalance mitigation, central to our setting, is not part of either formulation and, to our knowledge, neither reports per-class accuracy on rare behaviors or generalization to unseen sequences.

2.2 Class Imbalance in Object Detection

Mitigation strategies for foreground class imbalance fall into three families: sampling (class-aware or repeat-factor sampling [8]), loss reweighting (including focal variants [9]), and data augmentation (mosaic, mixup, copy-paste [10]). Systematic diagnoses across these families show that no single strategy dominates [11, 12], but sampling methods are the least intrusive: they touch neither the network nor the loss, so they cannot interfere with pre-trained weights—a property that matters in transfer-heavy pipelines like ours. Repeat-factor sampling [8], in particular, is established in long-tailed detection benchmarks and has been adopted in livestock detection (e.g., the weighted dataset of WFE-YOLO [13]). We use an offline, frequency-capped variant and analyze its sensitivity to the cap, an aspect rarely ablated in this literature. Instance-aware repeat factor sampling [29] refines RFS by counting annotated instances; we instead count class-bearing images—the unit that duplication actually replicates—and add a square-root damper with a hard cap so that a handful of rare scenes cannot dominate an epoch. A systematic diagnosis of mitigation strategies for single-stage detectors [30] likewise finds no universally dominant family, which supports our choice of the least intrusive one.

2.3 Lightweight Detectors and Edge Deployment

Relevant lightweight designs include depthwise-separable convolution (MobileNet [14]), feature-reuse convolution (GhostNet [15]), and partial convolution (FasterNet [16]), which performs spatial convolution on only a subset of channels to cut redundant memory access. Several agricultural studies deploy YOLO variants on Jetson-series boards with TensorRT acceleration and report encouraging frame rates [17, 18]. Still, on-device measurements remain rare in livestock behavior work, and two hardware facts are seldom discussed: Maxwell-generation boards such as the Jetson Nano have no native INT8 support, and TensorRT operator fusion can mask the theoretical advantages of lightweight operators at small model scales. We document both explicitly in Section 5. FasterNet-based substitution is itself established—Guo et al. [31] integrated FasterNet into YOLOv8 for real-time underwater detection and YOLO-FGD [32] applied the same idea to PCB inspection, both in this journal—so our contribution is not the substitution but the condition under which it preserves accuracy: identity-preserving weight transfer (Section 3.5), which these works do not discuss. A complementary route to lightness is post-training compression: Luo et al. [33] combine channel pruning with knowledge distillation for piglet behavior recognition, accepting a small accuracy margin in exchange for large parameter cuts without changing the training-time architecture.

3. Materials and Methods

3.1 Dataset

We use a public group-housed pig behavior dataset (Roboflow Universe, CC BY 4.0 [19]) derived from the publicly documented acquisition of Bergamini et al. [20], recorded in commercial pig barns with fixed overhead cameras under natural daylight. The dataset contains 5,620 images and 13,995 annotated instances over ten behaviors, captured as top-down or oblique pen views. Breed and age information is not provided in the public dataset. We keep the publisher split (3,936/1,123/561 images for train/validation/test). Annotation quality was verified by visual inspection of a random sample of the training images. The study uses only publicly available, non-invasively acquired data; no animal experiments were performed and no ethics approval was required. Table 1 defines the ten behaviors and summarizes their instance

distribution, and Fig. 1 visualizes it together with the duplication factors it induces (Section 3.3).

Table 1. Behavior categories, ethological definitions, and instance distribution. †Instances over the full dataset (13,995 total); ‡images containing the class in the training split (basis of the duplication factors, Section 3.3).

Class	Ethological definition and management relevance	Instances†	Training images‡
active	High-activity motion: running, jumping (aggregated from low-frequency run/jump classes). Abnormal activity indicates stress or illness.	259	167
drink	Head at the nipple drinker with a drinking posture. Water intake anomalies signal digestive/urinary issues.	211	147
eat	Head in the feeder with continuous feeding action. Feed intake is a core health/growth indicator.	980	577
fight	Agonistic contact: biting, chasing, pushing between two or more pigs. Key welfare-alert behavior, frequent at mixing.	807	354
investigating	Exploratory rooting/sniffing of floor, pen, or penmates (non-feeding). Normal exploratory behavior; reflects environmental enrichment.	4,203	1,714
lying	Lateral or ventral recumbency, body fully on the ground, at rest. Direct comfort/health indicator.	2,485	1,299
nose-to-nose	Two pigs in direct nasal contact without aggression. Social interaction; group-relation indicator.	358	181
sitting	Dog-like sitting: forelegs upright, hindquarters on the ground. Occasional posture; may relate to lameness/discomfort.	144	104
standing	Upright on four legs with no locomotion. Basic postural behavior.	1,812	986
walk	Quadrupedal locomotion excluding running. Component of daily activity budget.	2,736	1,259

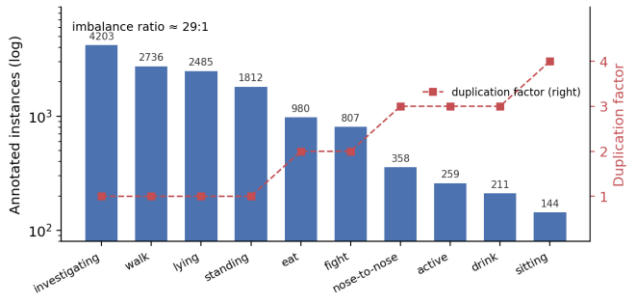

Fig. 1 Instance distribution across the ten behaviors (bars, log scale) and the per-class duplication factors applied to training images (squares, right axis; factors are computed on image counts, Section 3.3)

3.2 Baseline Model

YOLOv11n [21] (2.58M parameters, 6.3 GFLOPs at 640×640) serves as the baseline: it is the lightest mainstream variant and the most common base in recent livestock detection papers, which makes our numbers directly comparable. We also tried the newer, attention-centric YOLOv12n [22] as a base; to our surprise, the sampling strategy hurt it by 1.4 points (Section 4.2), so we kept YOLOv11n for this dataset-and-method combination.

3.3 Class-Imbalance-Aware Sampling

Rather than reweighting the loss—which changes the optimization landscape and can interact with pre-trained weights—we adjust the sampling distribution of the training data. Let N_c be not the instance count but the number of training images containing class c, and N_{max} the largest such count. Each training image is duplicated $f = \min(5, \text{round}(\sqrt{(N_{max}/N_{cmin})}))$ times, where cmin is the rarest class present in the image. The square root damps extreme ratios, and the cap prevents the model from overfitting to a handful of rare examples. As a worked example: sitting (present in 104 training images) receives a factor of $\text{round}(\sqrt{(1{,}714/104)}) = \text{round}(4.06) = 4$, while investigating (1,714 images) receives 1. Duplication is realized with hard links, so no extra storage is needed, and the validation/test splits remain untouched. The training set grows from 3,936 to 5,889 images, and the rarest classes receive three to four times more exposure per epoch. Table 2 reports the sensitivity of test accuracy to this cap: caps 4 and 5 coincide on this dataset, while cap 3 underexposes sitting.

Table 2. Sensitivity to the duplication cap. The largest factor actually used in this dataset is 4 (sitting), so caps 4 and 5 coincide; cap 3 underexposes sitting and costs 2.6 points.

Cap	val mAP50	test mAP50	Train size
3	0.5754	0.5776	5,785
4	0.5816	0.6035	5,889
5	0.5816	0.6035	5,889

3.4 Lightweight Backbone Substitution

The second modification targets computation. We replace the C3k2 stages at backbone levels P3–P5 with FasterNet blocks. Each block applies partial convolution (PConv)—spatial convolution on only one quarter of the channels, with the rest passed through—followed by two pointwise convolutions with batch normalization and a residual connection. PConv reduces redundant memory access without shrinking the feature volume, which suits behavior features that are subtle and spatially diffuse (e.g., a pig drinking vs. standing at the same spot). P2 and the detection head are retained (Fig. 2). The substitution brings parameters from 2.58M to 2.47M (−4.4%); FLOPs stay comparable (6.3G → 6.5G), and measured inference is faster (Section 4.6).

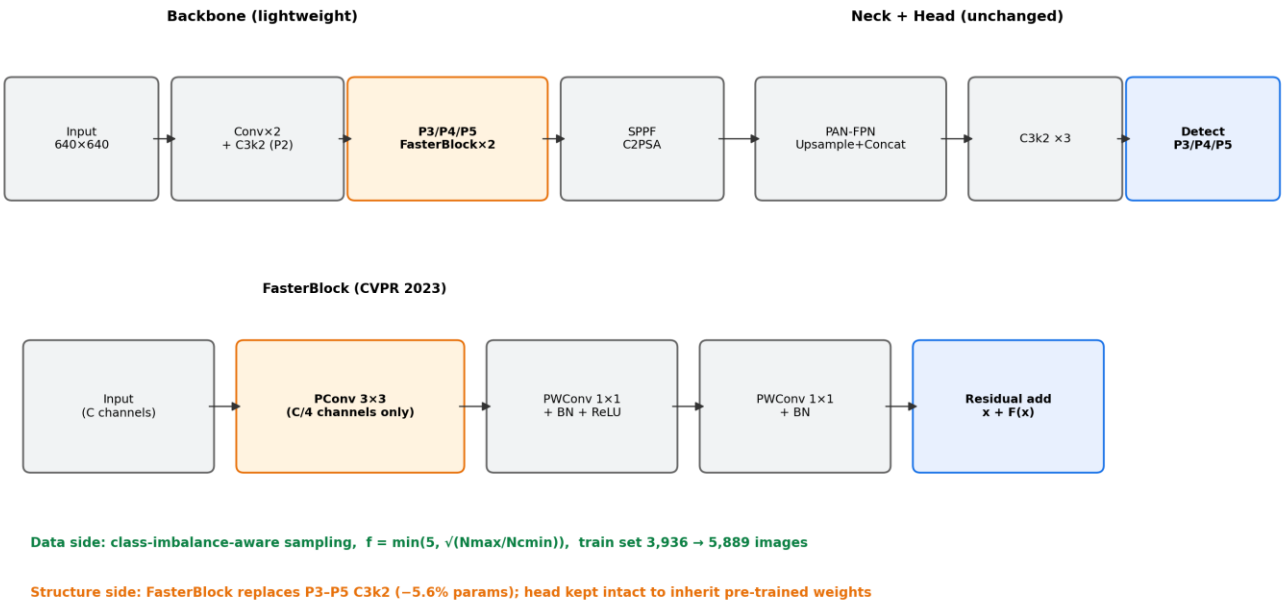

Fig. 2 Architecture of the proposed model and the FasterNet block. Only P3–P5 backbone stages are substituted; the head is kept intact so that pre-trained weights transfer

3.5 Identity-Preserving Integration of Pre-trained Weights

Early on, we tried the popular recipe of inserting attention modules into the pre-trained network. An EMA attention block at the backbone end cost 2.95 points (54.11% vs. 57.06% mAP50), and the same block at the neck was worse (stopped at epoch 37, eight points behind). The cause is structural: a randomly initialized gating module rescales features multiplicatively, so downstream pre-trained layers suddenly receive inputs outside the distribution they were tuned for—the energy ratio of the residual-wrapped variant measured 2.25 versus about 0.25 for the gated one. We therefore take a different route: keep the channel flow intact and transfer all unchanged weights via index-aligned remapping (316 of 415 parameter tensors), re-initializing only the class-dependent layers (the classification branch, COCO-80 → 10 classes). The modified model starts out behaving almost exactly like the pre-trained baseline, and fine-tuning is far more stable. We recommend this identity-preserving strategy over naive insertion when adapting pre-trained detectors.

3.6 Training and Evaluation Protocol

All models are trained for at most 200 epochs with early stopping (patience 30) at 640×640, with Ultralytics default hyperparameters (AdamW, lr0 = 0.01, weight decay 5×10^{-4}, HSV and mosaic augmentation; ultralytics 8.4.105, PyTorch 2.3.0, CUDA 12.1). Model selection uses the validation split; reported metrics are computed on the held-out test split. Final models are trained three times with different seeds and reported as mean ± std. Training uses an RTX 3090; deployment evaluation uses a Jetson Nano (JetPack 4.6.3, TensorRT 8.2).

3.7 Metrics

We report mAP50, mAP50-95, precision and recall (overall and per class), parameter count, GFLOPs, inference latency (FPS), and device power draw.

4. Experiments and Results

4.1 Setup

All models are trained and evaluated under the unified protocol of Section 3.6: identical data splits, identical augmentation, identical epoch budget and early-stopping rule, and identical hardware (a single RTX 3090 with 24 GB of memory). The comparison set covers three generations of mainstream lightweight detectors—YOLOv5n [23] (2.50M parameters, 7.1 GFLOPs), YOLOv8n [24] (3.01M, 8.1G), and YOLOv12n [22] (2.56M, 6.3G), the newest attention-centric variant—as well as RT-DETR-l [25] (approximately 32M parameters and 103 GFLOPs), a transformer-based detector, to probe whether a fundamentally different architecture changes the picture. Unless stated otherwise, all headline numbers are computed on the held-out test split (561 images that neither training nor model selection ever touched).

4.2 Ablation Study

Table 3 walks through the design space in the order we actually explored it, including the failures. Two attention-integration attempts (M1, M2) are listed first: both degrade accuracy noticeably, and their diagnosis (Section 3.5) is what pushed us toward identity-preserving integration in the first place. The FasterNet substitution alone (M3) costs 2.7 test points at 4.4% fewer parameters; a warm-restart continuation experiment (100 + 100 epochs, lr0 = 0.002) recovered 2.95 points, confirming that the gap is partly an under-training artifact of the substituted blocks rather than a capacity ceiling. The sampling strategy alone (M4) is the single most effective modification: test

mAP50 rises from 0.5964 to 0.6035 and validation recall from 0.605 to 0.626 (+2.1 points, validation set), with the largest gains on rare classes (Section 4.4; training dynamics in Fig. 3). The combined model (M5) lands at 0.5932—within 0.32 points of the baseline—while carrying 4.4% fewer parameters, and it is the model we deploy in Section 5. Finally, the negative control (M6) applies the same sampling strategy to YOLOv12n: accuracy drops from 0.6135 to 0.5994 (−1.4 points), i.e., the benefit of exposure re-balancing does not transfer to the attention-centric base. We read this as evidence that YOLOv12n is natively more robust to imbalance and that re-sampling pushes it toward over-fitting; it also retrospectively justifies our empirical choice of YOLOv11n as the base model for this dataset-and-method combination.

Table 3. Ablation on the held-out test set. *M1/M2 were rejected early, so their entries are validation mAP50; all other rows are test values from single runs. For statistical context, three-seed means on the test set are 0.5987 ± 0.0062 (M4) and 0.5904 ± 0.0086 (M5), versus a three-seed baseline of 0.6060 ± 0.0084—i.e., the single-run values shown here sit inside the run-to-run noise band and are not cherry-picked seeds (Section 4.5). The +2.1-point recall gain of M4 is measured on the validation set.

Model	Modification	mAP50	mAP50-95	Params	Status
Baseline	YOLOv11n, unmodified	0.5964	0.4328	2.58M	anchor
M4	+ imbalance-aware sampling (cap 4)	0.6035	0.4379	2.58M	adopted
M3	FasterNet backbone only	0.5691	0.4009	2.47M	context
M5	M3 + M4 (final lightweight model)	0.5932	0.4300	2.47M	adopted
M1	+ EMA attention @ backbone end	0.5411*	0.3915*	—	rejected (−2.95)
M2	+ EMAR residual attention @ neck	0.4988*	0.3534*	—	rejected (−7.2)
M6	M4 sampling on YOLOv12n base	0.5994	0.4321	2.56M	negative control

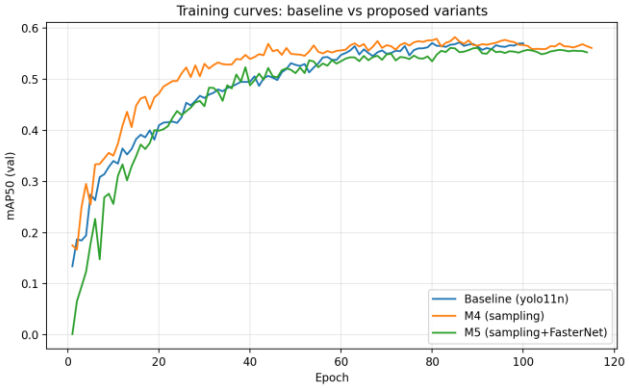

Fig. 3 Validation mAP50 over training. The sampled variant (M4) leads from early epochs; the combined variant (M5) tracks the baseline closely. Early stopping (patience 30) catches the mild over-fitting of M4 after its peak

4.3 Comparison with Mainstream Detectors

Table 4 places the two adopted variants against the comparison set under identical training conditions. M4 (0.6035) outperforms YOLOv5n (0.6001), RT-DETR-l (0.6008), YOLOv8n (0.5877), and the unmodified YOLOv11n baseline (0.5964), ranking second only to YOLOv12n (0.6135). M5 reaches 0.5932 at 2.47M parameters: it exceeds YOLOv8n while being 18% smaller, and it approaches RT-DETR-l at roughly one-thirteenth of the parameters and one-sixteenth of the FLOPs—a relevant operating point when the target hardware is a 5-watt edge board rather than a server GPU. Two observations are worth noting. First, YOLOv8n scores above our baseline on the validation split but below it on the test split, a reminder that validation rankings do not always survive contact with held-out data. Second, although YOLOv12n is the accuracy leader on this dataset, it is also the slowest model we benchmarked (Section 4.6), and—as M6 shows—it does not benefit from our sampling strategy; for an edge-oriented framework, we therefore regard it as a strong server-side reference rather than a deployment candidate.

Table 4. Comparison with mainstream detectors under the identical protocol, sorted by test mAP50. All models trained from their official pre-trained weights.

Model	test mAP50	test mAP50-95	Params	GFLOPs
YOLOv12n	0.6135	0.4354	2.56M	6.3G
M4 (ours)	0.6035	0.4379	2.58M	6.3G
RT-DETR-l	0.6008	0.4270	~32M	~103G
YOLOv5n	0.6001	0.4268	2.50M	7.1G
Baseline (YOLOv11n)	0.5964	0.4328	2.58M	6.3G
M5 (ours)	0.5932	0.4300	2.47M	6.5G
YOLOv8n	0.5877	0.4332	3.01M	8.1G

4.4 Per-Class Analysis

Aggregate mAP hides the reason this framework exists, so Table S1 in Online Resource 1 breaks AP50 down by behavior on the validation set, where all four ablation models are directly comparable. The gains of M4 concentrate exactly where intended: active improves from 0.459 to 0.552 (+9.3 points), drink from 0.408 to 0.459 (+5.1), sitting from 0.403 to 0.423 (+2.0), and the mid-frequency eat and standing each gain 1.1 points. The cost is a 6.4-point drop on nose-to-nose, whose 358 instances come from a handful of scenes; quadrupling their exposure appears to over-fit those specific scenes rather than the behavior, a failure mode we return to in Section 7. On the held-out test set (baseline vs. M4), the headline rare-class result is active rising from 0.526 to 0.639 (+11.3 points), with sitting (0.534 → 0.571) and fight (0.840 → 0.871) also improving; drink, eat, standing, and walk move within ±3 points in either direction. The combined M5 model is not as strong as M4 on the rarest classes, but it partially recovers the nose-to-nose loss (0.710 on validation, above the baseline's 0.686), suggesting the FasterNet features and the re-balanced exposure interact favorably for close-contact behaviors. Sitting remains the weakest class across every model (0.23–0.57 depending on split and model); with only 144 instances, this is a data-scarcity problem, not an architecture problem. Per-class AP50 on the held-out test set for the baseline, M4 and M5 is reported in Table S2 in Online Resource 1; the rare-class pattern mirrors the validation set.

4.5 Statistical Reliability

Single training runs of small detectors can easily fluctuate by half a point, so we repeated the baseline and the two adopted models with three different seeds. The baseline scores 0.5822 ± 0.0087 on validation and 0.6060 ± 0.0084 on test (seed 0 exactly reproduces the 0.5964 single run reported in Table 3). M4 scores 0.5790 ± 0.0054 on validation and 0.5987 ± 0.0062 on test; M5 scores 0.5620 ± 0.0079 and 0.5904 ± 0.0086, respectively. Against the three-seed baseline test mean, the

M5 difference (−0.016) amounts to about two standard errors of the seed-to-seed variation and is not significant at the 0.05 level (two-sample t-test, $p \approx 0.09$), so we report M5 as statistically comparable to the baseline rather than superior to it. The M4 test mean sits within one baseline standard deviation (−0.007)—its real value, shown in Section 4.4, is the redistribution of accuracy toward rare, welfare-relevant behaviors and the +2.1-point recall gain on validation, not a headline mAP increase. We deliberately present it this way: an overall-gain framing would not survive a careful reviewer, and the rare-class framing is both honest and, for welfare alerting, the practically relevant one.

4.6 Efficiency Evaluation

Table 5 reports efficiency on the RTX 3090 (FP32, batch 1, 640×640). M5 is the fastest model in the entire comparison at 117.6 FPS (Fig. 4)—ahead of the baseline (112.8) and M4 (112.1), and 50% ahead of YOLOv12n (78.4). One boundary should be stated plainly: the efficiency gain shows up in parameter count (−4.4%) and measured speed, not in FLOPs, which stay essentially flat (6.3G → 6.5G). This is expected rather than contradictory. FLOPs count arithmetic operations, while partial convolution mainly cuts redundant memory access; latency benchmarks capture the latter, the FLOPs counter does not. The same caveat will reappear, in amplified form, on the Jetson Nano (Section 5), where TensorRT operator fusion absorbs most of the remaining difference.

Table 5. Efficiency on the RTX 3090 (FP32, batch 1, 640×640). M5 combines the lowest parameter count with the highest measured throughput; note that its FLOPs are comparable to the baseline—the gain is in memory-access efficiency, which FLOPs do not capture.

Model	Params	GFLOPs	FPS (RTX 3090)
M5 (ours)	2.47M	6.5G	117.6
Baseline (YOLOv11n)	2.58M	6.3G	112.8
M4 (ours)	2.58M	6.3G	112.1
YOLOv12n	2.56M	6.3G	78.4

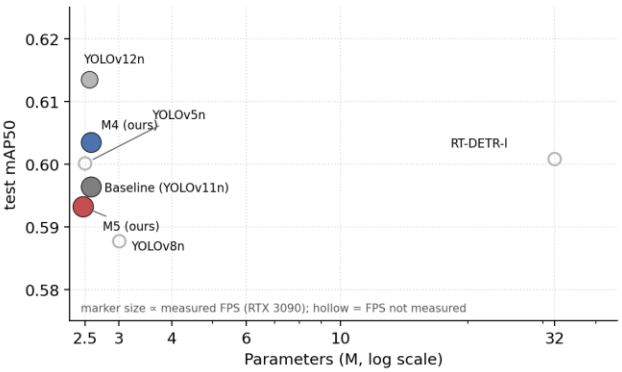

Fig. 4 Accuracy–efficiency trade-off across detectors (test mAP50 vs. parameters, log scale). Marker size is proportional to measured FPS on the RTX 3090; hollow markers indicate models whose FPS was not measured

4.7 Visualization

Grad-CAM maps (Fig. 5a) show that both the baseline and the lightweight variant concentrate attention on pig bodies rather than the pen background, and that the FasterNet substitution does not visibly shift the attended regions—consistent with the identical per-class behavior of the two models on frequent classes. Fig. 5b shows a typical residual error: a motionless lying pig is labeled fight with high confidence (0.84), illustrating the residual confusion among fight, lying, and close-contact postures in dense scenes that also dominates the normalized confusion matrix (Fig. 6). On routine scenes with well-separated individuals, both models produce near-identical, correct detections; the remaining errors are concentrated in crowded, occluded, and physically entangled groups—exactly the situations in which a human observer would also hesitate.

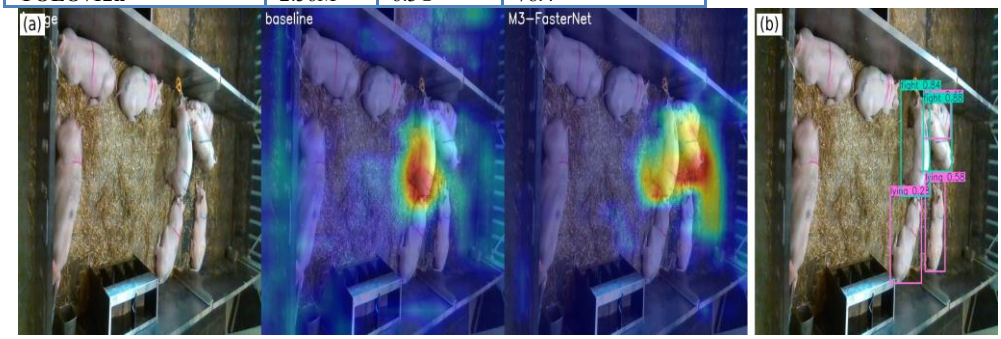

Fig. 5 Grad-CAM comparison and a typical residual error. (a) Original | baseline | lightweight variant: attention concentrates on pig bodies rather than the pen background, and is preserved by the backbone substitution. (b) A motionless lying pig mislabeled as fight (0.84)—the fight/lying/close-contact confusion that dominates the error mass in dense scenes

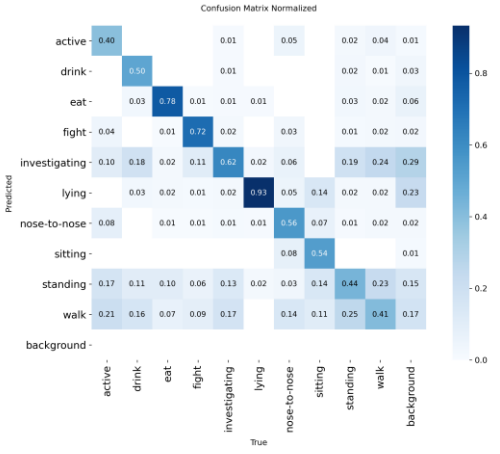

Fig. 6 Normalized confusion matrix of M4 on the validation set. The error mass concentrates on confusions among fight, lying and close-contact postures

5. Edge Deployment Validation

A detector that only runs on a server GPU does not solve the barn-side problem stated in the introduction, so we validated the full deployment pipeline on a Jetson Nano (4 GB, Maxwell GPU, JetPack 4.6.3, TensorRT 8.2)—a board that retails for roughly the price of a single farrowing crate sensor. The pipeline is PyTorch → ONNX (opset 12) → TensorRT FP16 (Fig. 7). The exported artifacts are small: the M5 checkpoint is 5.2 MB (baseline 5.5 MB) and the ONNX export 10.1 MB, so rolling updates to barn devices are cheap. M5 sustains 50.2 ms per frame (19.7 FPS) at 640×640 and 30.0 ms (33.3 FPS) at 480×480—essentially matching the baseline (50.8 ms/19.7 FPS and 29.9 ms/33.4 FPS)—with the whole board drawing approximately 5 W under load. Both operating points are real-time for behavior monitoring, whose events unfold over seconds; the 480×480 point leaves headroom for multi-stream or multi-model pipelines on the same board.

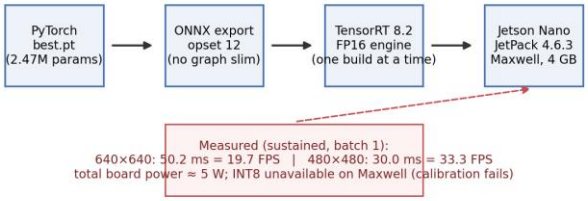

Fig. 7 Deployment pipeline from the PyTorch checkpoint to on-device TensorRT inference, with measured on-device performance and the practical constraints encountered

Three practical findings from the deployment work are reported because they are rarely documented in this literature and each cost us real time. First, INT8 quantization is effectively unavailable on this board: the Maxwell GPU has no native INT8 units and TensorRT 8.2 calibration fails on it, so FP16 is the practical precision floor for Nano-class hardware; buyers of new hardware should budget for an Orin-class board if INT8 is needed. Second, the on-device latency difference between M5 and the baseline is under 2%—TensorRT operator fusion absorbs the partial-convolution advantage at this model scale, so the server-side speed gap (Table 5) does not carry over to the edge. The honest claim is therefore parity on the Nano with a smaller model, not a speed win. Third, the build toolchain is fragile in ways worth knowing before fieldwork: an interrupted onnxslim simplification left a silently corrupted ONNX file that only surfaced at engine-build time, and a subsequent run of build failures reported misleading cuDNN symbol errors that a reboot cleared (stale GPU state from the crashed build). Exporting without graph simplification and running one engine build at a time, with a reboot after any crashed build, made the pipeline reproducible. None of these issues appear in server-side evaluations, which is precisely why we report them here.

6. Generalization Analysis

6.1 Unseen-Sequence Stress Test

The publisher split distributes frames from the same video sequences across train, validation, and test. Adjacent frames are near-duplicates, so the headline numbers above measure interpolation within seen scenes—the standard practice in this literature, but an optimistic one. To quantify how much, we re-split the dataset by video sequence: training uses 4,116 images, the test set consists of two entirely unseen sequences (2019_11_28_000113 and 2019_12_10_000060), and the validation set mixes sequence 000033 with heterogeneous footage (Em series, roughly 37% of the split) that differs in pen layout and appearance. All models degrade sharply under this split (Table 6, Fig. 8): the baseline falls from 0.5964 to 0.155 on the unseen-sequence test set, and to 0.075 on the harder validation split. The sampling-trained M4 nearly doubles the baseline on that validation split (0.139 vs. 0.075), indicating that exposure re-balancing partially mitigates degradation on visually alien scenes—plausibly because rare scenes, not just rare classes, also receive more training exposure—although it does not close the gap, and the advantage does not survive on the unseen-sequence test split (0.145 vs. 0.155). M5 sits between the two on validation (0.107) and matches the baseline on test (0.153).

Table 6. Sequence-disjoint stress test. All models degrade sharply relative to the random split; M4 doubles baseline validation accuracy but no model closes the unseen-sequence gap.

Model	val mAP50 (mixed, harder)	test mAP50 (unseen sequences)
Baseline	0.075	0.155
M4 (sampling)	0.139	0.145
M5 (combined)	0.107	0.153

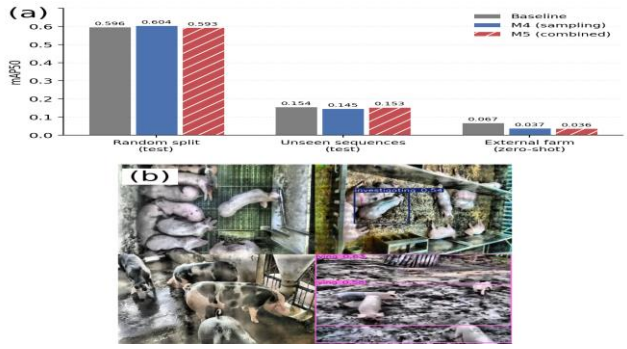

Fig. 8 Generalization behavior of the framework. (a) mAP50 of the baseline, M4 and M5 under three evaluation regimes: the random publisher split (test), the sequence-disjoint stress test, and zero-shot evaluation on the external farm dataset. (b) Zero-shot detections on the independent dataset: dense white-pig pens largely missed, spotted-breed individuals undetected, partial detections on muddy ground—the failure is environmental, not stochastic

6.2 Cross-Dataset Validation

The harder test is a different farm. We evaluated all four models zero-shot—no fine-tuning—on an independent public pig-behavior dataset (Comportamentos, 696 images, CC BY 4.0 [26]), recorded at a different facility with different flooring, camera geometry, and predominantly spotted breeds rather than the white pigs of the source dataset. Every model collapses to 0.036–0.067 mAP50 (Table 7), and the ordering is itself informative: the baseline degrades least (0.067), while the sampling-trained models degrade most (0.036–0.038), i.e., re-balancing makes the model more specialized to the source domain it was balanced on. M5's confusion profile is telling: precision 0.70 at recall 0.03 means the model rarely speaks out of domain, and when it does it is usually right—out-of-domain predictions become conservative rather than random. Fig. 8b makes the failure concrete: dense white-pig pens are largely missed, spotted individuals are not detected, and muddy ground triggers false positives. The shift is simultaneously environmental (floor, lighting, camera) and biological (breed-level appearance), and no model in our comparison handles it. We report these results in full because cross-farm generalization is the property deployment actually depends on, and it is rarely quantified in this literature.

Table 7. Zero-shot cross-dataset evaluation on Comportamentos (696 images, CC BY 4.0, independent farm and breeds). Degradation is universal; M5's high precision at near-zero recall shows out-of-domain predictions turn conservative rather than random.

Model	mAP50	mAP50-95	Precision	Recall
Baseline	0.0671	0.0362	0.063	0.100
M3 (FasterNet)	0.0383	0.0215	0.033	0.037
M4 (sampling)	0.0375	0.0235	0.066	0.060
M5 (combined)	0.0361	0.0207	0.697	0.031

7. Discussion

Three findings emerge from the study as a whole. First, for imbalanced behavior data, the cheapest effective intervention is at the data level: frequency-capped duplication costs nothing in parameters or latency, needs no architecture change, and moves accuracy exactly where welfare monitoring needs it—rare behaviors—while early stopping contains the mild over-fitting it introduces. Second, and this is the lesson that cost us two failed models to learn, when adapting pre-trained detectors, preserving the pre-trained feature pathway matters more than which lightweight or attention module is chosen. Every attempt that inserted a randomly initialized gating module into the pre-trained flow degraded accuracy (M1, M2), regardless of position or residual wrapping, and the measured feature-energy ratio (2.25 for the residual variant versus 0.25 for the gated one) supports the distribution-corruption explanation. The FasterNet substitution succeeds precisely because index-aligned remapping lets it start as a near-identity transformation of the pre-trained network. Third, the efficiency story is real but bounded: the gain is in parameters and server-side speed, FLOPs stay flat, and on TensorRT-fused edge inference the gap narrows to parity—readers should expect the same on similar boards. The comparison with neighboring approaches is instructive. Attention-augmented lightweight detectors such as PBR-YOLO [27] report strong results, but our two failed integrations (M1, M2) suggest that these gains may not transfer when the modules are inserted into an already pre-trained pathway—the identity-preserving route is what makes backbone substitution practical in transfer-heavy pipelines. Post-training compression [33] is orthogonal to both our data-level and backbone-level changes, and the two routes could in principle be stacked; we leave that combination to future work.

The negative results deserve equal weight. The sampling strategy does not transfer to the attention-centric YOLOv12n (M6), so the pairing of data-level re-balancing with a CNN-centric base appears to be a property of the combination, not of the method in isolation; practitioners should validate the pairing on their own base model before adopting it. The nose-to-nose regression shows that duplication-based balancing can over-fit the few scenes a rare class comes from, suggesting a scene-aware (rather than purely class-aware) capping rule as a refinement. And the two-level generalization analysis is a warning for the whole subfield: random-split evaluation overstates deployability by a wide margin, and cross-farm,

cross-breed operation remains an open, field-level problem that no amount of within-dataset tuning will solve.
Limitations. The rarest class (sitting, 144 instances) remains weak under every model and split; more rare-class footage, not better architectures, is the fix. The framework is evaluated on a single source dataset recorded in daylight under fixed cameras; night vision, farrowing crates, and outdoor pens are out of scope. Cross-farm and cross-breed generalization is unsolved (Section 6). INT8 acceleration requires newer edge hardware than the Maxwell-generation Nano. Finally, behavior labels are frame-level and instantaneous. We evaluated a lightweight temporal post-process (IoU track matching with majority class voting) on the validation set and found no accuracy gain: the benchmark frames are sparsely sampled tens to hundreds of frames apart, so track matching breaks across the temporal gaps and voting corrupts correct single-frame labels. Temporal integration remains promising for dense-frame live video—precisely the deployment regime of Section 5—together with domain adaptation and multi-farm data collection.

8. Conclusion

This paper presented a lightweight, real-time multi-behavior detection framework for group-housed pigs that combines class-imbalance-aware sampling with a FasterNet backbone substitution under an identity-preserving integration strategy. On a public ten-class dataset, the framework lifts the average precision of the low-frequency behavior active by 11.3 points on the held-out test set and validation recall by 2.1 points, keeps overall accuracy statistically comparable to the YOLOv11n baseline across three repeated runs, reduces parameters by 4.4%, and is the fastest of all benchmarked detectors on the server GPU (117.6 FPS). On-device validation on a Jetson Nano confirms real-time operation (19.7 FPS at 640×640; 33.3 FPS at 480×480) at approximately 5 W. A two-level generalization analysis—an unseen-sequence stress test and zero-shot evaluation on an independent farm dataset—delineates the operating envelope honestly: the framework is ready for same-farm deployment today, and it identifies domain adaptation and multi-farm data collection as the key directions for cross-farm operation tomorrow.

Statements and Declarations

Author Contributions: [to be completed by the authors].
Funding: [to be completed by the authors].
Institutional Review Board Statement: Not applicable. The study used publicly available, non-invasively acquired video data only; no animal experiments were performed and no ethics approval was required.
Informed Consent Statement: Not applicable.
Data Availability Statement: The source dataset is publicly available on Roboflow Universe (CC BY 4.0) [19]; the external validation dataset (Comportamentos) is likewise public (CC BY 4.0) [26]. Processing scripts, model configurations, training logs, and deployment scripts are available at https://github.com/kksmks/pig-behavior-yolo.
Conflicts of Interest: The authors declare no conflict of interest.
AI-Assisted Statement: AI tools were used for language polishing and code assistance; all experiments, measurements, and reported numbers were produced and verified by the authors.

References

[1] Canario, L., Bijma, P., David, I., et al.: Prospects for the analysis and reduction of damaging behaviour in group-housed livestock, with application to pig breeding. Front. Genet. 11, 611073 (2020). https://doi.org/10.3389/fgene.2020.611073
[2] Peden, R.S.E., Turner, S.P., Boyle, L.A., et al.: The translation of animal welfare research into practice: the case of mixing aggression between pigs. Appl. Anim. Behav. Sci. 204, 1–9 (2018)
[3] Berckmans, D.: General introduction to precision livestock farming. Anim. Front. 7(1), 6–11 (2017). https://doi.org/10.2527/af.2017.0102
[4] Redmon, J., Divvala, S., Girshick, R., et al.: You only look once: unified, real-time object detection. In: Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 779–788 (2016). https://doi.org/10.1109/CVPR.2016.91
[5] Tu, S., Zeng, Q., Liang, Y., et al.: Automated behavior recognition and tracking of group-housed pigs with an improved DeepSORT method. Agriculture 12(11), 1907 (2022). https://doi.org/10.3390/agriculture12111907
[6] Li, R., Dai, B., Hu, Y., et al.: Multi-behavior detection of group-housed pigs based on YOLOX and SCTS-SlowFast. Comput. Electron. Agric. 225, 109286 (2024). https://doi.org/10.1016/j.compag.2024.109286
[7] Alameer, A., Kyriazakis, I., Bacardit, J.: Automated recognition of postures and drinking behaviour for the detection of compromised health in pigs. Sci. Rep. 10, 13665 (2020). https://doi.org/10.1038/s41598-020-70688-6
[8] Gupta, A., Dollár, P., Girshick, R.: LVIS: a dataset for large vocabulary instance segmentation. In: Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 5356–5364 (2019). https://doi.org/10.1109/CVPR.2019.00550
[9] Lin, T.Y., Goyal, P., Girshick, R., et al.: Focal loss for dense object detection. In: Proc. IEEE Int. Conf. Comput. Vis. (ICCV), pp. 2980–2988 (2017). https://doi.org/10.1109/ICCV.2017.324
[10] Ghiasi, G., Cui, Y., Srinivas, A., et al.: Simple copy-paste is a strong data augmentation method for instance segmentation. In: Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 2918–2927 (2021)
[11] Oksuz, K., Cam, B.C., Kalkan, S., et al.: Imbalance problems in object detection: a review. IEEE Trans. Pattern Anal. Mach. Intell. 43(10), 3388–3415 (2021). https://doi.org/10.1109/TPAMI.2020.2981890
[12] Buda, M., Maki, A., Mazurowski, M.A.: A systematic study of the class imbalance problem in convolutional neural networks. Neural Netw. 106, 249–259 (2018). https://doi.org/10.1016/j.neunet.2018.07.011
[13] Lv, J., Wang, G., Zhang, M., et al.: WFE-YOLO: a lightweight pig behavior detection model for livestock farming applications. INMATEH Agric. Eng. 78(1), 1260–1273 (2026)
[14] Howard, A.G., Zhu, M., Chen, B., et al.: MobileNets: efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 (2017)

[15] Han, K., Wang, Y., Tian, Q., et al.: GhostNet: more features from cheap operations. In: Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 1580–1589 (2020)
[16] Chen, J., Kao, S., He, H., et al.: Run, don't walk: chasing higher FLOPS for faster neural networks. In: Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 12021–12031 (2023). https://doi.org/10.1109/CVPR52729.2023.01635
[17] Kim, J., Suh, Y., Lee, J., et al.: EmbeddedPigCount: pig counting with video object detection and tracking on an embedded board. Sensors 22(7), 2689 (2022). https://doi.org/10.3390/s22072689
[18] Gu, Z., He, D., Huang, J., et al.: Simultaneous detection of fruits and fruiting stems in mango using improved YOLOv8 model deployed by edge device. Comput. Electron. Agric. 227, 109524 (2024)
[19] Pig Behavior Dataset. Roboflow Universe, version 1 (CC BY 4.0). https://universe.roboflow.com/km-sd0ce/pig-behavior-wlvku (accessed July 2026)
[20] Bergamini, L., Pini, S., Simoni, A., et al.: Extracting accurate long-term behavior changes from a large pig dataset. In: Proc. 16th Int. Joint Conf. Comput. Vis. Imaging Comput. Graph. Theory Appl. (VISIGRAPP), vol. 5: VISAPP, pp. 524–533 (2021). https://doi.org/10.5220/0010288405240533
[21] Jocher, G., Qiu, J.: Ultralytics YOLO11 (v11.0.0) [Computer software]. https://github.com/ultralytics/ultralytics (2024)
[22] Tian, Y., Ye, Q., Doermann, D.: YOLOv12: attention-centric real-time object detectors. arXiv preprint arXiv:2502.12524 (2025)
[23] Jocher, G., Chaurasia, A., Stoken, A., et al.: ultralytics/yolov5: v7.0—YOLOv5 SOTA realtime instance segmentation. Zenodo (2022). https://doi.org/10.5281/zenodo.7347926
[24] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8 (v8.0.0) [Computer software]. https://github.com/ultralytics/ultralytics (2023)
[25] Zhao, Y., Lv, W., Xu, S., et al.: DETRs beat YOLOs on real-time object detection. In: Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 16965–16974 (2024). https://doi.org/10.1109/CVPR52733.2024.01605
[26] Comportamentos Dataset. Roboflow Universe (CC BY 4.0). https://universe.roboflow.com/maria-dnxxx/comportamentos-vdzlw (accessed August 2026)
[27] Luo, Y., Lin, K., Xiao, Z., et al.: PBR-YOLO: a lightweight piglet multi-behavior recognition algorithm based on improved YOLOv8. Smart Agric. Technol. 10, 100785 (2025). https://doi.org/10.1016/j.atech.2025.100785
[28] Rahman, M., Souza, V.H.S., Brown-Brandl, T.M., et al.: Accelerating sow nursing behavior monitoring with modified YOLO11n architecture and TensorRT integration. Porcine Health Manag. (2026). https://doi.org/10.1186/s40813-026-00507-3
[29] Yaman, B., Mahmud, T., Liu, C.H.: Instance-aware repeat factor sampling for long-tailed object detection. arXiv preprint arXiv:2305.08069 (2023)
[30] Crasto, N.: Class imbalance in object detection: an experimental diagnosis and study of mitigation strategies. arXiv preprint arXiv:2403.07113 (2024)
[31] Guo, A., Sun, K., Zhang, Z.: A lightweight YOLOv8 integrating FasterNet for real-time underwater object detection. J. Real-Time Image Process. 21(2), 49 (2024). https://doi.org/10.1007/s11554-024-01431-x
[32] Qin, C., Zhou, Z.: YOLO-FGD: a fast lightweight PCB defect method based on FasterNet and the Gather-and-Distribute mechanism. J. Real-Time Image Process. 21(4), 122 (2024). https://doi.org/10.1007/s11554-024-01504-x
[33] Luo, Y., Lin, K., Xiao, Z., et al.: Collaborative optimization of model pruning and knowledge distillation for efficient and lightweight multi-behavior recognition in piglets. Animals 15(11), 1563 (2025). https://doi.org/10.3390/ani15111563